

Laboratorium 2 – Klasy i obiekty w nowoczesnym C++

Made with the help of AI

Cel zajęć: poznanie podstawowej składni klas w nowoczesnym C++ (oraz dla tych, którzy poznali wcześniej Javę – różnic). Ćwiczymy definiowanie klas, pola, metody, poziomy dostępu, enkapsulację, konstruktory, destruktory oraz obserwowanie działania konstruktora kopiującego, konstruktora przenoszącego i operatorów przypisania.

Założenie: koncepcja programowania obiektowego jest już znana (np. z Javy lub Pythona). Na tych zajęciach nie uczymy od zera, czym jest obiekt, ale skupiamy się na tym, jak zapisuje się klasy w C++ i jakie mechanizmy są charakterystyczne dla tego języka.

Polecenie organizacyjne: Do każdego zadania napisz program, skompiluj go, uruchom i przetestuj. Tam, gdzie w konstruktorach albo operatorach przypisania pojawiają się komunikaty, zwróć uwagę, w którym momencie programu są wypisywane.

Wskazówki ogólne

1. **Klasa w C++ nie musi być w osobnym pliku.** Na potrzeby laboratorium możesz pisać klasę i funkcję `main()` w jednym pliku `.cpp`. W większych projektach klasy zwykle dzieli się na pliki `.h/.hpp` i `.cpp`.
2. **Domyślny poziom dostępu w `class` to `private`.** To ważna różnica składniowa. Jeżeli zapomnisz napisać `public:`, metody również będą prywatne.
3. **Nie używamy jeszcze wskaźników.** W przykładach korzystamy z typów takich jak `std::string` i `std::vector`, aby skupić się na składni klas i mechanizmach kopiowania oraz przenoszenia.
4. **Nowoczesny styl C++:**
 - inicjalizuj pola bezpośrednio w klasie,
 - używaj listy inicjalizacyjnej konstruktora,
 - stosuj `std::string` zamiast tablic znaków,
 - stosuj `std::move`, gdy chcesz jawnie przenieść wartość,
 - zapisuj funkcje w stylu `auto nazwa(...) -> typ`.
5. **Uwaga na terminologię.** W C++ mówimy o **destruktorze**, a nie o „dekonstruktorze”. Destruktor uruchamia się, gdy obiekt kończy swoje życie.
6. **Nie wszystko trzeba pisać ręcznie.** W prawdziwym kodzie C++ często pozwalamy kompilatorowi wygenerować konstruktor kopiujący, konstruktor przenoszący, operatory przypisania i destruktory. Dzisiaj jednak napiszemy je ręcznie, aby zobaczyć, kiedy są wywoływane.

Krótko: Java a C++

W tych zajęciach zakładamy, że idea klasy i obiektu jest już znana. Najważniejsze jest więc zauważenie różnic składniowych i semantycznych między Javą a C++.

Java	C++
Zmienna obiektowa zwykle przechowuje referencję do obiektu.	Obiekt może być zwykłą zmienną, np. <code>Task t{"Nauka"};</code> .
Usuwanie obiektów zajmuje się garbage collector.	Destruktor wywołuje się deterministycznie, gdy obiekt kończy życie.
Przypisanie obiektów zwykle kopiuje referencję.	Przypisanie może uruchomić kopiujący albo przenoszący operator przypisania.
Nie ma bezpośredniego odpowiednika <code>std::move</code> .	<code>std::move</code> pozwala potraktować obiekt jako możliwy do przeniesienia.
Modyfikator dostępu zwykle stoi przy polu lub metodzie.	W klasie C++ stosujemy sekcje <code>public:</code> , <code>private:</code> , <code>protected:</code> .

Najważniejsza intuicja: w C++ bardzo często interesuje nas cykl życia obiektu: kiedy powstaje, kiedy jest kopiowany, kiedy jest przenoszony i kiedy jest niszczone.

Szablon startowy

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>

using std::cout;
using std::endl;
using std::move;
using std::string;
using std::vector;

auto main() -> int
{
    // tutaj wpisz kod
    return 0;
}
```

1. Prosta definicja klasy

Najprostsza klasa może zawierać pola, czyli dane obiektu, oraz metody, czyli funkcje wykonywane na obiekcie.

```
class Student
{
public:
    string name{};
    int semester{1};

    auto print() const -> void
    {
        cout << "Student: " << name
              << ", semestr: " << semester << endl;
    }
};

auto main() -> int
{
    Student s{};
    s.name = "Anna";
    s.semester = 2;
    s.print();

    return 0;
}
```

Co warto zauważyć:

- obiekt tworzymy podobnie jak zmienną: `Student s{};`,
- do publicznych pól i metod odwołujemy się przez kropkę: `s.name`, `s.print()`,
- metoda `print()` ma dopisek `const`, ponieważ nie zmienia obiektu.

2. `public`, `private` i `protected`

W C++ poziomy dostępu zapisujemy wewnątrz klasy jako etykiety zakończone dwukropkiem.

```
class Example
{
public:
    // Dostępne z zewnątrz klasy.
    auto public_method() -> void {}

private:
    // Dostępne tylko wewnątrz tej klasy.
    int secret_value{42};

protected:
    // Dostępne wewnątrz tej klasy i w klasach dziedziczących.
}
```

```
    int value_for_children{10};  
};
```

Na tych zajęciach najczęściej używamy:

- **private** dla pól klasy,
- **public** dla metod, z których można korzystać z zewnątrz,
- **protected** tylko sygnalizacyjnie – dokładniej wrócimy do niego przy dziedziczeniu.

3. Atrybuty klasy i metody

W nowoczesnym C++ pola klasy można inicjalizować od razu w miejscu deklaracji. Dzięki temu obiekt ma sensowny stan początkowy nawet wtedy, gdy użyjemy domyślnego konstruktora.

```
class Rectangle  
{  
private:  
    double width{1.0};  
    double height{1.0};  
  
public:  
    auto area() const -> double  
    {  
        return width * height;  
    }  
  
    auto print() const -> void  
    {  
        cout << "width = " << width  
              << ", height = " << height  
              << ", area = " << area() << endl;  
    }  
};  
  
auto main() -> int  
{  
    Rectangle r{};  
    r.print();  
  
    return 0;  
}
```

W tym przykładzie pola są prywatne, więc nie można napisać:

```
r.width = 10.0; // blad: width jest private
```

4. Enkapsulacja, settery i gettery

Enkapsulacja polega na ukrywaniu wewnętrznego stanu obiektu i udostępnianiu kontrolowanego dostępu przez metody. Dzięki temu klasa może pilnować poprawności danych.

```
class BankAccount
{
private:
    string owner{"unknown"};
    double balance{0.0};

public:
    auto set_owner(string new_owner) -> void
    {
        if (!new_owner.empty())
        {
            owner = new_owner;
        }
    }

    auto get_owner() const -> string
    {
        return owner;
    }

    auto get_balance() const -> double
    {
        return balance;
    }

    auto deposit(double amount) -> void
    {
        if (amount > 0.0)
        {
            balance += amount;
        }
    }

    auto withdraw(double amount) -> bool
    {
        if (amount > 0.0 && amount <= balance)
        {
            balance -= amount;
            return true;
        }

        return false;
    }
};
```

Dlaczego nie udostępniamy pola **balance** jako **public**? Gdyby saldo było publiczne,

użytkownik klasy mógłby ustawić wartość ujemną bez żadnej kontroli. Dzięki metodom `deposit()` i `withdraw()` klasa pilnuje swoich zasad.

5. Konstruktor i lista inicjalizacyjna

Konstruktor to specjalna metoda uruchamiana podczas tworzenia obiektu. W C++ często używamy **listy inicjalizacyjnej**, czyli fragmentu po dwukropku.

```
class Course
{
private:
    string name{"unknown"};
    int ects{0};

public:
    Course(string course_name, int course_ects)
        : name{course_name}, ects{course_ects}
    {
        cout << "Konstruktor Course" << endl;
    }

    auto print() const -> void
    {
        cout << name << " (ECTS: " << ects << ")" << endl;
    }
};

auto main() -> int
{
    Course cpp{"Programowanie w C++", 5};
    cpp.print();

    return 0;
}
```

Warto zapamiętać: pola są inicjalizowane przed wejściem do ciała konstruktora. Dlatego lista inicjalizacyjna jest zwykle lepsza niż przypisywanie wartości wewnątrz konstruktora.

6. Kopiowanie, przenoszenie, przypisanie i destruktor

W C++ obiekty mogą być kopiowane albo przenoszone. To ważna różnica względem Javy, gdzie zmienne obiektowe zwykle przechowują referencje do obiektów. W C++ obiekt może być zwykłą zmienną i może mieć własny cykl życia.

Poniższa klasa wypisuje komunikaty, aby pokazać, która metoda specjalna jest uruchamiana.

```
class Note
{
private:
    string title{"untitled"};
```

```
vector<string> lines{};

public:
    Note()
    {
        cout << "Konstruktor domyslny" << endl;
    }

    Note(string note_title)
        : title{move(note_title)}
    {
        cout << "Konstruktor z argumentem" << endl;
    }

    Note(const Note& other)
        : title{other.title}, lines{other.lines}
    {
        cout << "Konstruktor kopiujacy" << endl;
    }

    Note(Note&& other) noexcept
        : title{move(other.title)}, lines{move(other.lines)}
    {
        cout << "Konstruktor przenoszacy" << endl;
    }

    auto operator=(const Note& other) -> Note&
    {
        cout << "Kopiujacy operator przypisania" << endl;

        if (this != &other)
        {
            title = other.title;
            lines = other.lines;
        }

        return *this;
    }

    auto operator=(Note&& other) noexcept -> Note&
    {
        cout << "Przenoszacy operator przypisania" << endl;

        if (this != &other)
        {
            title = move(other.title);
            lines = move(other.lines);
        }

        return *this;
    }
}
```

```
}

~Note()
{
    cout << "Destruktor: " << title << endl;
}

auto add_line(string line) -> void
{
    lines.push_back(move(line));
}

auto print() const -> void
{
    cout << "Notatka: " << title << endl;
    for (const auto& line : lines)
    {
        cout << "- " << line << endl;
    }
}

};
```

Przykład użycia klasy **Note**

```
auto create_note() -> Note
{
    Note temp{"Tymczasowa"};
    temp.add_line("Ta notatka powstała w funkcji.");
    return temp;
}

auto main() -> int
{
    cout << "--- Tworzenie obiektu ---" << endl;
    Note a{"C++"};
    a.add_line("Klasy mogą mieć pola i metody.");

    cout << "--- Kopiowanie przy tworzeniu ---" << endl;
    Note b{a};

    cout << "--- Kopiujące przypisanie ---" << endl;
    Note c{"Pusta"};
    c = a;

    cout << "--- Przenoszenie przy tworzeniu ---" << endl;
    Note d{move(a)};

    cout << "--- Przenoszące przypisanie ---" << endl;
    Note e{"Inna"};
    e = create_note();
}
```

```
    cout << "--- Koniec programu ---" << endl;  
    return 0;  
}
```

Pytania do obserwacji:

- Kiedy pojawia się komunikat o konstruktorze kopiującym?
- Kiedy pojawia się komunikat o operatorze przypisania?
- Czy destruktor uruchamia się tylko raz?
- Co zmienia użycie `move(a)`?
- Czy wynik może zależeć od optymalizacji kompilatora?

Uwaga: współczesne kompilatory mogą pomijać niektóre kopie dzięki optymalizacji. Jeżeli chcesz łatwiej obserwować kopiowanie i przenoszenie, możesz kompilować program poleceniem podobnym do:

```
g++ -std=c++20 -Wall -Wextra -pedantic -fno-elide-constructors main.cpp -o main
```

Zadania

Zadanie 1. Pierwsza klasa **Book**

Napisz klasę **Book**, która reprezentuje książkę.

Klasa powinna mieć publiczne pola:

- **title** typu **string**,
- **author** typu **string**,
- **year** typu **int**.

Dodaj metodę:

```
auto print() const -> void
```

która wypisze informacje o książce.

Wskazówki:

- Zacznij od prostego przykładu z polami publicznymi.
- Utwórz w **main()** dwa obiekty klasy **Book**.
- Ustaw wartości pól i wywołaj metodę **print()**.

Zadanie 2. Prywatne pola w klasie **Book**

Przerób klasę **Book** tak, aby pola były prywatne.

Dodaj metody:

- **set_title(string new_title)**,
- **set_author(string new_author)**,
- **set_year(int new_year)**,
- **get_title()**,
- **get_author()**,
- **get_year()**.

Wskazówki:

- W setterze dla roku sprawdź, czy rok jest większy od 0.
- W setterach dla napisów sprawdź, czy napis nie jest pusty.
- Sprawdź, co stanie się po próbie bezpośredniego dostępu do pola prywatnego.

Zadanie 3. Domyślne wartości atrybutów

Napisz klasę `Movie`, która ma prywatne pola:

- `title` z wartością domyślną `"unknown"`,
- `duration_minutes` z wartością domyślną `0`,
- `rating` z wartością domyślną `0.0`.

Dodaj metodę `print()` i odpowiednie settery.

Wskazówki:

- Utwórz obiekt `Movie m{}`; bez ustawiania pól.
- Wywołaj `print()` i sprawdź wartości domyślne.
- Następnie ustaw poprawne wartości i wypisz obiekt ponownie.

Zadanie 4. Konstruktor klasy `Product`

Napisz klasę `Product`, która ma prywatne pola:

- `name`,
- `price`,
- `quantity`.

Dodaj konstruktor:

```
Product(string product_name, double product_price, int product_quantity)
```

który używa listy inicjalizacyjnej.

Dodaj metody:

- `print()`,
- `total_value()`, która zwraca `price * quantity`.

Wskazówki:

- W konstruktorze wypisz komunikat: `"Konstruktor Product"`.
- Utwórz kilka produktów w `main()`.
- Wypisz wartość całkowitą każdego produktu.

Zadanie 5. Enkapsulacja w klasie `Wallet`

Napisz klasę `Wallet`, która przechowuje prywatne pole:

```
double money{0.0};
```

Dodaj metody:

- `deposit(double amount)`,
- `spend(double amount)`,
- `get_money()`.

Metoda `spend()` powinna zwracać `bool`: `true`, jeśli operacja się udała, albo `false`, jeśli zabrakło środków.

Wskazówki:

- Nie pozwalaj wpłacać kwot ujemnych.
- Nie pozwalaj wydać więcej, niż znajduje się w portfelu.
- Przetestuj kilka poprawnych i niepoprawnych operacji.

Zadanie 6. Klasa `Task` z konstruktorem kopiującym

Napisz klasę `Task`, która ma prywatne pola:

- `name` typu `string`,
- `done` typu `bool` z wartością domyślną `false`.

Dodaj:

- konstruktor z argumentem `string`,
- konstruktor kopiujący,
- metodę `mark_done()`,
- metodę `print()`.

W konstruktorze kopiującym wypisz komunikat:

```
cout << "Konstruktor kopiujacy Task" << endl;
```

Wskazówki:

- Utwórz obiekt `Task a{"Nauczyc sie klas"};`
- Następnie utwórz kopię: `Task b{a};`
- Zmień stan jednego z obiektów i sprawdź, czy drugi obiekt zmienił się automatycznie.

Zadanie 7. Kopiujący operator przypisania

Rozbuduj klasę `Task` z poprzedniego zadania o kopiujący operator przypisania:

```
auto operator=(const Task& other) -> Task&
```

Wewnątrz operatora wypisz komunikat:

```
cout << "Kopiujacy operator przypisania Task" << endl;
```

Wskazówki:

- Pamiętaj o sprawdzeniu `if (this != &other)`.
- Zwróć `*this`.
- Przetestuj różnicę między:

```
Task b{a}; // konstruktor kopiujacy  
b = a;     // operator przypisania
```

Zadanie 8. Konstruktor przenoszący

Rozbuduj klasę `Task` o konstruktor przenoszący:

```
Task(Task&& other) noexcept
```

Wewnątrz wypisz komunikat:

```
cout << "Konstruktor przenoszacy Task" << endl;
```

Wskazówki:

- Do przeniesienia napisu użyj `move(other.name)`.
- Pole `done` możesz po prostu skopiować, ponieważ jest typu `bool`.
- Przetestuj kod:

```
Task a{"Zrobic laboratorium"};  
Task b{move(a)};
```

- Po przeniesieniu nie zakładaj, jaka dokładnie wartość napisu pozostała w obiekcie `a`. Możesz go zniszczyć albo przypisać mu nową wartość.

Zadanie 9. Przenoszący operator przypisania

Rozbuduj klasę `Task` o przenoszący operator przypisania:

```
auto operator=(Task&& other) noexcept -> Task&
```

Wewnątrz wypisz komunikat:

```
cout << "Przenoszacy operator przypisania Task" << endl;
```

Wskazówki:

- Pamiętaj o sprawdzeniu `if (this != &other)`.
- Zwróć `*this`.
- Przetestuj kod:

```
Task a{"Pierwsze zadanie"};  
Task b{"Drugie zadanie"};  
b = move(a);
```

Zadanie 10. Destruktor i cykl życia obiektu

Dodaj do klasy `Task` destruktora:

```
~Task()  
{  
    cout << "Destruktor Task: " << name << endl;  
}
```

Następnie napisz program, w którym tworzysz obiekty w różnych blokach kodu.

```
auto main() -> int  
{  
    cout << "Start programu" << endl;  
  
    Task a{"Obiekt z main"};  
  
    {  
        Task b{"Obiekt z bloku"};  
        b.print();  
    }  
  
    cout << "Koniec programu" << endl;  
    return 0;  
}
```

Wskazówki:

- Sprawdź, kiedy niszczone jest obiekt `b`.
- Sprawdź, kiedy niszczone jest obiekt `a`.
- Porównaj to z życiem obiektów znanym z Javy.

Zadanie 11. Klasa **Playlist** z wektorem

Napisz klasę **Playlist**, która ma prywatne pola:

- `name` typu `string`,
- `songs` typu `vector<string>`.

Dodaj:

- konstruktor z nazwą playlisty,
- metodę `add_song(string song)`,
- metodę `print()`,
- konstruktor kopiujący z komunikatem,
- konstruktor przenoszący z komunikatem,
- kopiujący operator przypisania z komunikatem,
- przenoszący operator przypisania z komunikatem,
- destruktor z komunikatem.

Wskazówki:

- W konstruktorze przenoszącym użyj `move(name)` i `move(songs)`.
- Utwórz playlistę, dodaj kilka piosenek, skopiuj ją i przenieś.
- Obserwuj komunikaty wypisywane przez program.

Zadanie 12. Funkcja zwracająca obiekt

Do klasy **Playlist** dodaj funkcję niezależną od klasy:

```
auto create_example_playlist() -> Playlist
```

Funkcja powinna utworzyć playlistę, dodać kilka utworów i zwrócić gotowy obiekt.

Wskazówki:

- Wywołaj funkcję w `main()`.
- Przypisz wynik funkcji do nowego obiektu.
- Przypisz wynik funkcji do już istniejącego obiektu.
- Porównaj, kiedy uruchamia się konstruktor przenoszący, a kiedy przenoszący operator przypisania.
- Dla lepszej obserwacji możesz użyć flagi `-fno-elide-constructors`.

Sekcja dodatkowa A. Eksperymenty

Ta sekcja nie jest obowiązkowa na zajęciach. Jej celem jest spokojne zaobserwowanie, kiedy C++ tworzy kopię, kiedy wykonuje przeniesienie, a kiedy tylko przypisuje wartość do istniejącego obiektu.

```
Task a{"A"};
Task b{a};           // konstruktor kopiujacy

Task c{"C"};
c = a;               // kopiujacy operator przypisania

Task d{move(a)};     // konstruktor przenoszacy

Task e{"E"};
e = move(c);         // przenoszacy operator przypisania
```

Pytania do eksperymentu:

- Który komunikat pojawił się w której linijce?
- Czym różni się utworzenie nowego obiektu od przypisania do już istniejącego obiektu?
- Co zmieni się po usunięciu `move`?
- Co zmieni się po kompilacji z flagą `-fno-elide-constructors`?
- Czy destruktory wywołują się w odwrotnej kolejności niż konstruktory?

Sekcja dodatkowa B. Przewidź wynik programu

Najpierw nie uruchamiaj programu. Zapisz na kartce przewidywaną kolejność komunikatów. Dopiero potem skompiluj i uruchom kod.

```
auto make_task() -> Task
{
    Task temp{"Z funkcji"};
    return temp;
}

auto main() -> int
{
    cout << "1. Start" << endl;

    Task a{"A"};
    Task b{a};

    cout << "2. Przed blokiem" << endl;
    {
        Task c{"C"};
        c = b;
    }
    cout << "3. Po bloku" << endl;
```

```
Task d{make_task()};

cout << "4. Koniec main" << endl;
return 0;
}
```

Po uruchomieniu odpowiedz:

- Czy wynik był taki, jak przewidywano?
- Który obiekt został zniszczony jako pierwszy?
- Czy kompilator pominął jakieś kopiowanie albo przenoszenie?
- Jak zmienił się wynik po dodaniu flagi `-fno-elide-constructors`?

Czego dzisiaj jeszcze nie robimy

Niektóre ważne tematy z C++ zostawiamy na kolejne zajęcia. Dzisiaj tylko przygotowujemy grunt.

- Nie zarządzamy ręcznie pamięcią.
- Nie używamy `new` i `delete`.
- Nie omawiamy jeszcze wskaźników.
- Nie piszemy własnych kontenerów.
- Nie omawiamy jeszcze pełnej zasady Rule of Five ani Rule of Zero jako stylu projektowania.
- Nie omawiamy jeszcze SOLID – na razie budujemy podstawy przez enkapsulację i odpowiedzialność klasy za własny stan.

Podsumowanie: Checklista po laboratorium

Po zajęciach sprawdź, czy potrafisz odpowiedzieć na poniższe pytania.

- Czy umiem stworzyć prostą klasę w C++?
- Czy wiem, co oznaczają sekcje `public`, `private` i `protected`?
- Czy wiem, dlaczego pola najczęściej powinny być prywatne?
- Czy potrafię napisać prosty setter i getter?
- Czy wiem, kiedy działa konstruktor kopiujący?
- Czy wiem, czym różni się `Task b{a};` od `b = a;`?
- Czy wiem, kiedy działa konstruktor przenoszący?
- Czy wiem, kiedy działa przenoszący operator przypisania?

- Czy wiem, że `std::move` samo niczego magicznie nie przenosi, tylko pozwala użyć operacji przenoszącej?
- Czy potrafię zaobserwować moment wywołania destruktora?